レイキャスト 3D ゲームの数学

1. はじめに：このドキュメントについて

このドキュメントでは、レイキャストという、3D ゲームを構成する技術で使われている数学を、中学校で習う数学までを前提に説明します。

レイキャストとは？

「レイ」は英語で「光線」という意味です。プレイヤーの目から光線を飛ばして、壁にぶつかったところを計算することで、3D のように見える画面を作ります。1990 年代のゲーム「Wolfenstein 3D」や「DOOM」で使われた技術です。

必要な数学の知識

- 一次関数（直線の式）
- 三角関数（sin, cos, tan）
- 三平方の定理（ピタゴラスの定理）

それでは、順番に見ていきましょう！

2. 一次関数（直線の式）

2.1. 直線ってなに？

まっすぐな線のことを「直線」と言います。グラフ上で直線を表すには、数式を使います。

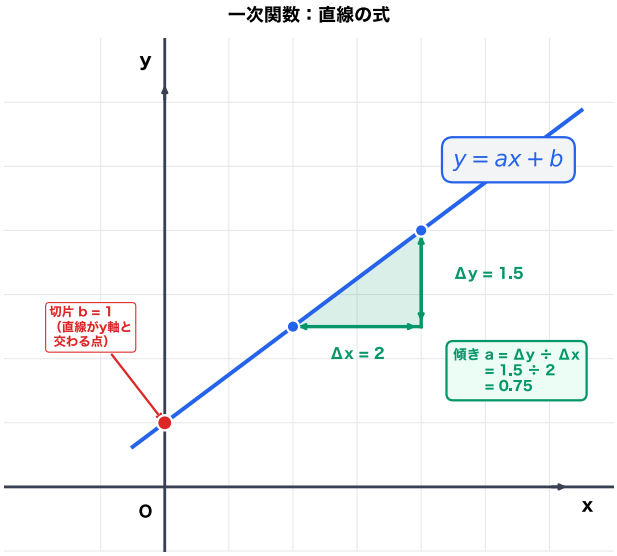

図 1: 一次関数のグラフ。横軸が x、縦軸が y

2.2. 直線の式：y = ax + b

直線を表す基本の式は次のとおりです：

$$y = ax + b$$

この式に出てくる文字の意味を説明します。

傾き（a）

「傾き」は、直線がどれくらい斜めになっているかを表す数字です。

- a が正の数 → 右上がりの線
- a が負の数 → 右下がりの線
- a = 0 → 水平な線

具体的には、「x が 1 増えたとき、y がどれだけ増えるか」を表しています。

例えば、a = 2 なら「x が 1 増えると、y は 2 増える」ということです。

切片（b）

「切片」は、直線が y 軸（x = 0 の線）と交わる点の y 座標です。

例えば、b = 3 なら、直線は点(0, 3)を通ります。

2.3. 2 点から傾きを求める

2 つの点 (x_1, y_1) と (x_2, y_2) がわかっているとき、傾きは次の式で求められます：

$$a = \frac{y_2 - y_1}{x_2 - x_1}$$

これは「y の変化量÷x の変化量」という意味です。

例題

点 A(1, 2)と点 B(3, 6)を通る直線の傾きは？

$$a = \frac{6-2}{3-1} = \frac{4}{2} = 2$$

答え：傾きは 2

2.4. レイキャストでの使い方

レイ（光線）は「ある点から伸びる直線」です。レイキャストでは、直線の傾きの代わりに「角度」を使います。

角度と傾きの関係は、次の節で説明する三角関数の `tan` で表されます。

3. 三角関数（sin, cos, tan）

3.1. 三角関数ってなに？

三角関数は、「角度」を「長さ」に変換するための道具です。

ゲームでは、プレイヤーの向いている方向を「角度」で持っています。でも、実際に移動するには「x 方向に何ピクセル、y 方向に何ピクセル動くか」という「長さ」が必要です。

この変換をしてくれるのが三角関数です。

3.2. 単位円で理解する

「単位円」とは、半径が 1 の円のことです。この円を使うと、三角関数の意味がわかりやすくなります。

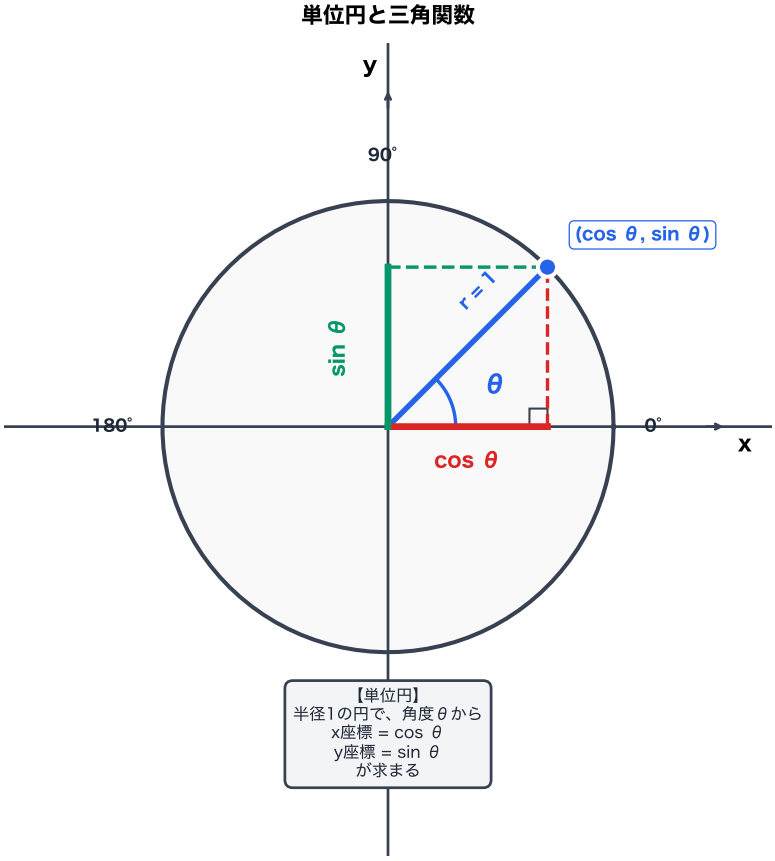

図 2: 単位円。中心が原点(0,0)で、半径が 1 の円

円の中心を原点(0, 0)に置きます。そして、x 軸の正の方向（右方向）を角度 0° として、反時計回りに角度を測ります。

角度 θ のとき、円周上の点の座標は：

- x 座標 = cos(θ)
- y 座標 = sin(θ)

つまり、**cos は「横方向の成分」**、**sin は「縦方向の成分」**を表しています。

3.3. cos, sin, tan の意味

cos（コサイン）

角度 θ の方向を向いたとき、「横方向にどれだけ進むか」を表します。

- cos(0°) = 1（右を向くと、横方向に 1 進む）
- cos(90°) = 0（上を向くと、横方向には進まない）

sin（サイン）

角度 θ の方向を向いたとき、「縦方向にどれだけ進むか」を表します。

- sin(0°) = 0（右を向くと、縦方向には進まない）
- sin(90°) = 1（上を向くと、縦方向に 1 進む）

tan（タンジェント）

直線の「傾き」を表します。sin を cos で割った値です。

$$\tan(\theta) = \frac{\sin(\theta)}{\cos(\theta)}$$

3.4. 角度の単位：度とラジアン

角度には 2 つの表し方があります。

度（°）

日常でよく使う単位。1 周 = 360°

ラジアン（rad）

プログラミングでよく使う単位。1 周 = 2π（約 6.28）

変換の目安：

- 90° = π/2 ラジアン（約 1.57）
- 180° = π ラジアン（約 3.14）
- 360° = 2π ラジアン（約 6.28）

JavaScript の `Math.sin()` や `Math.cos()` はラジアンを使います。

3.5. ゲームでの使い方

移動の計算

プレイヤーが角度 θ の方向に「速さ v」で移動するとき：

- x 方向の移動量 = v × cos(θ)
- y 方向の移動量 = v × sin(θ)

横移動（ストレイフ）

横に移動したいときは、向いている方向を 90° 回転させます：

- 横方向の x 成分 = cos(θ + 90°)
- 横方向の y 成分 = sin(θ + 90°)

4. レイキャストの仕組み

4.1. レイキャストとは

レイキャストは、「レイ（光線）を飛ばして、壁との交点を見つける」方法です。

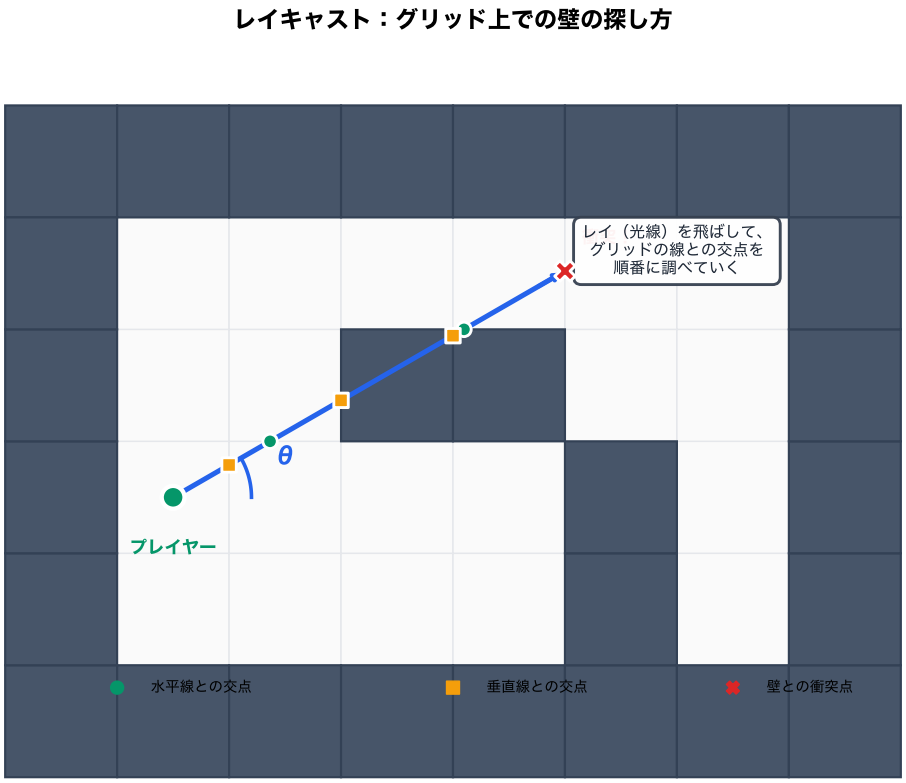

図 3: マス目（グリッド）上をレイが進んでいく様子

マップはマス目（グリッド）になっています。レイがグリッドの線と交わる点を順番に調べていき、最初に壁にぶつかる場所を探します。

4.2. なぜグリッドの線を調べるの？

壁はマス目の単位で配置されています。つまり、壁があるかどうかが変わるのは、グリッドの線を越えたときだけです。

だから、グリッドの線との交点だけを調べれば効率的に壁を見つけられます。

4.3. 水平線との交点を探す

まず、横方向に引かれたグリッドの線（水平線）との交点を探します。

レイの出発点を (x_0, y_0)、角度を θ とすると、水平線 y = k との交点は：

$$x = x_0 + \frac{k - y_0}{\tan(\theta)}$$

この式は、「y がどれだけ変化したかから、x の変化を逆算する」という意味です。

4.4. 垂直線との交点を探す

次に、縦方向に引かれたグリッドの線（垂直線）との交点を探します。

垂直線 x = k との交点は：

$$y = y_0 + (k - x_0) \times \tan(\theta)$$

4.5. 壁を見つける手順

1. 水平線との交点を順番に調べて、壁にぶつかったら止まる
2. 垂直線との交点を順番に調べて、壁にぶつかったら止まる
3. 水平線でぶつかった壁と、垂直線でぶつかった壁を比べる
4. **近い方**を「最初にぶつかった壁」として採用する

4.6. 注意点：ゼロで割らない

tan(θ) が 0 に近いとき（θ が 0° や 180° に近いとき)、計算でゼロで割ることになってしまいます。

これを防ぐために、プログラムでは tan(θ) が 0 のときに、とても小さい値（0.00001 など）で代用します。

5. 3D の壁を描く

5.1. 遠近法の考え方

同じ大きさのものでも、近くにあると大きく見え、遠くにあると小さく見えます。これが「遠近法」です。

レイキャストでは、壁までの距離がわかったら、その距離に応じて壁の高さを計算します。

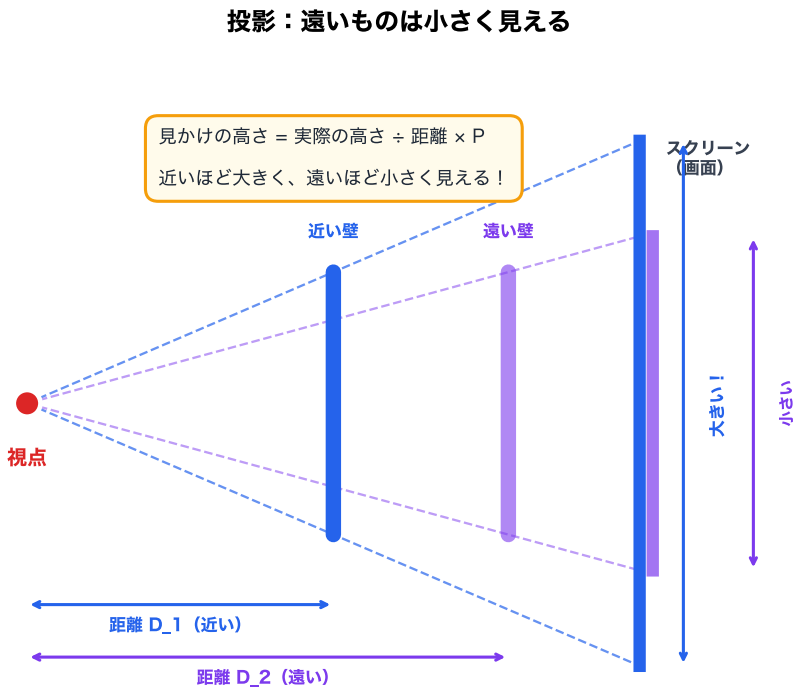

図 4: 投影の仕組み。遠いものほど小さく見える

5.2. 壁の高さの計算

見かけの壁の高さは、次の式で計算します：

$$H_{\text{見かけ}} = \left(\frac{H_{\text{実際}}}{D}\right) \times P$$

- $H_{\text{実際}}$ = 壁の実際の高さ（このゲームでは 1 マス分 = TILE_SIZE）
- D = 壁までの距離
- P = スクリーンまでの距離（後で説明）

この式は「距離が 2 倍になると、高さは半分になる」ということを表しています。

5.3. スクリーン距離（P）とは

「スクリーン距離」は、プレイヤーの目から仮想的なスクリーンまでの距離です。

視野角を f、画面の幅を w とすると：

$$P = \frac{w \div 2}{\tan(f \div 2)}$$

視野角が 60°、画面幅が 900 ピクセルなら：

$$P = \frac{450}{\tan(30°)} \fallingdotseq \frac{450}{0.577} \fallingdotseq 780$$

5.4. フィッシュアイ補正

ここで 1 つ問題があります。普通に計算すると、画面の端に行くほど壁が膨らんで見えてしまいます。これを「フィッシュアイ効果」と言います。

なぜ膨らむの？

画面の端のレイは、正面より斜めに飛んでいます。斜めに飛ぶと、同じ壁でも距離が長くなります。でも、人間の目で見たときは、その壁は正面と同じ距離にあるように見えるはずです。

補正の方法

レイの角度と正面の角度の差を Δ（デルタ）とすると、補正後の距離は：

$$D_{\text{補正後}} = D \times \cos(\Delta)$$

cos(Δ) を掛けることで、斜めに伸びた分を打ち消します。

6. まとめ

レイキャストで使う数学をまとめます。

一次関数

- 直線の式：y = ax + b
- 傾き a と切片 b で直線が決まる

三角関数

- cos(θ) = 横方向の成分
- sin(θ) = 縦方向の成分
- tan(θ) = 傾き = sin(θ) / cos(θ)

レイキャスト

- グリッドの線との交点を順番に調べる
- 水平線と垂直線を別々に調べて、近い方を採用

3D 描画

- 見かけの高さ = (実際の高さ / 距離) × スクリーン距離
- フィッシュアイ補正で歪みを防ぐ

これらの数学を組み合わせることで、2D のマップから 3D のような画面を作り出しています。中学校で習う数学が、実際のゲーム開発で役立っているのです！